SMS Spam Detector
Project Report

Submitted By:

Name: Ashutosh Bhutekar

Reg. No. : 25BAI11422

Course : Fundamentals in AI & ML

CONTENTS

- TITLE
- INTRODUCTION
- MOTIVATION OF THE PROJECT
- PROBLEM STATEMENT
- OBJECTIVE OF THE THE PROJECT
- EXISTING METHODS
- PROS AND CONS OF THE STATED METHODS
- HARDWARE AND SOFTWARE REQUIREMENTS
- METHODOLOGY AND GOAL
- FUNCTIONAL MODULES DESIGN AND ANALYSIS
- ALGORITHM DEVELOPMENT
- SOFTWARE ARCHITECTURAL DIAGRAM
- CODING
- OUTPUT
- KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM
- SIGNIFICANT PROJECT OUTCOMES
- TESTING AND REFINEMENT
- PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS
- CONTRIBUTION/ FINDINGS OF THE PROJECT
- LIMITATION/CONSTRAINTS OF THE SYSTEM
- CONCLUSIONS
- FUTURE ENHANCEMENTS
- REFERENCES

1. Introduction

We all get spam messages on our phones. Some of them are easy to ignore, the ones that say you won a lottery you never entered or ask you to click some random link. But I've noticed some of them are actually pretty convincing, and my grandmother almost entered her bank details on one of those fake sites last year.

I wanted to build something that could look at a text message and figure out whether it is spam or a normal message. It is just a small program that runs in the terminal. You type a message, it tells you spam or not spam.

The whole thing is built in Python. It uses a machine learning algorithm called Naive Bayes, which is one of the simplest classifiers out there but works really well for text-related problems. I trained it on a public dataset of real SMS messages.

---

2. Motivation of the Project

I picked this because it felt relatable. I get junk messages on my phone pretty regularly, maybe 2-3 a week. And like I mentioned, my grandmother almost got scammed last year through a text. She's not very tech-savvy and the message looked legit to her. That made me think about how this stuff actually works behind the scenes.

I was curious about how spam filters actually work. Like, what goes on behind the scenes when your phone automatically puts a message in the spam folder? I knew it had something to do with machine learning but I didn't really understand the details. So I figured building one myself would be the best way to learn.

---

3. Problem Statement

Spam messages are still a big problem in mobile communication. They range from annoying promotional texts to actually dangerous phishing attempts. While big companies like Google and Apple have built spam detection into their platforms, a lot of people still don't have access to good spam filtering, especially in developing countries or on basic phones.

The problem I'm trying to solve here is simple: given a text message, can we automatically tell if it is spam or not?

The input is a raw text message. The output is one of two labels – SPAM or NOT SPAM

---

4. Objective of the Project

What I wanted to achieve with this project:

- Build a working spam detector that runs in the command line
- Learn how to preprocess text data - cleaning, removing unnecessary words, stemming
- Understand how TF-IDF works and why it's useful for text problems
- Train a Naive Bayes model and see how well it performs on real data
- Get comfortable with the full ML workflow from raw data to predictions

5. Existing Methods

There are several ways people have approached spam detection over the years. The main ones I came across during my research are:

Rule-Based Filtering: This is the oldest method. Someone manually writes rules like "if the message contains the word FREE in all caps, mark it as spam." It works to some extent but it's very rigid. Spammers can easily get around it by changing their wording slightly.

Naive Bayes: This is what I used. It's a probability-based algorithm that looks at the words in a message and calculates the chance of it being spam based on what it learned from training data. It's simple, fast, and surprisingly accurate for text classification.

Support Vector Machine (SVM): SVM tries to draw a line (or a boundary in higher dimensions) between spam and non-spam messages. It generally gives slightly better accuracy than Naive Bayes but takes longer to train and is harder to understand.

Random Forest: This method builds a bunch of decision trees and combines their results. It's good for many types of problems but can sometimes overfit on text data.

Deep Learning (LSTM, BERT): These are the modern, heavy-duty approaches. They understand context and word meaning much better than any of the above methods. But they need huge amounts of data and powerful hardware to train. Way overkill for a project like this.

Keyword Blacklisting: It is the simplest approach. Just maintain a list of spam keywords and flag any message that contains them. It barely works in practice because spammers constantly change their vocabulary.

---

6. Pros and Cons of the Stated Methods

Rule-Based Filtering

- Pros: Easy to build, no training data needed, fast
- Cons: Can't handle new types of spam, needs constant manual updates, lots of false positives

Naive Bayes

- Pros: Fast, works well even with small data, easy to understand, great for text
- Cons: Assumes words are independent (which isn't always true), can struggle with words it hasn't seen before

SVM

- Pros: Often more accurate, handles high-dimensional data well
- Cons: Slower to train, harder to tune, not as intuitive

Random Forest

- Pros: Handles complex patterns, less sensitive to outliers
- Cons: Can overfit, slower than Naive Bayes, not the best fit for text

Deep Learning

- Pros: Best accuracy, understands context and meaning
- Cons: Needs tons of data, needs GPU, takes long to train, complete overkill for simple tasks

Keyword Blacklisting

- Pros: Dead simple to implement
- Cons: Terrible accuracy, very easy to bypass

I went with Naive Bayes because it made the most sense for this project. It's simple enough that I can fully understand what's happening under the hood, it trains in seconds, and the accuracy is genuinely good. For a fundamentals course project, I think using something I can explain properly matters more than squeezing out an extra 2% accuracy with a complex model.

---

7. Hardware and Software Requirements

Hardware:

- Any regular laptop or desktop — nothing special needed
- At least 4 GB RAM (though it would work with less)
- About 100 MB of free disk space
- No GPU needed at all
- Internet connection just for the initial setup (downloading packages and the dataset)

I built and tested this on my own laptop and it ran without any issues.

Software:

- Operating System: Linux, macOS, or Windows (I used Linux)
- Python 3.8 or higher
- pip (Python package manager)
- Libraries: pandas, scikit-learn, nltk
- Any text editor — I used VS Code
- Git for version control
- Terminal / Command Prompt

---

8. Methodology and Goal

Here's how I approached this project step by step:

Getting the data: I used the SMS Spam Collection dataset from UCI, which I downloaded from Kaggle. It has 5,572 SMS messages and each one is labeled as either ham or spam. It's a well-known dataset that's been used in a lot of research papers.

Looking at the data: I noticed that out of 5,572 messages, about 4,825 are ham and only 747 are spam. So there's a big imbalance, way more normal messages than spam. This is something I kept in mind throughout the project.

Cleaning the text: This was the part that took the most effort. Raw text messages are messy, they have random capitalization, punctuation, numbers, slang. I wrote a function that converts everything to lowercase, removes all the punctuation and numbers, gets rid of common filler words (like "the",

"is", "and"), and reduces words to their root form using something called stemming. So "running", "runs", and "ran" all become "run".

Converting text to numbers: Machine learning models work with numbers, not words. So I used a technique called TF-IDF to convert each cleaned message into a list of numbers. TF-IDF basically scores each word based on how important it is, words that appear a lot in one message but not across all messages get higher scores.

Splitting the data: I split the data into two parts - 80% for training and 20% for testing. The training data is what the model learns from. The test data is kept aside to check how well the model performs on messages it has never seen.

Training: I used the Multinomial Naive Bayes algorithm from scikit-learn. Training literally takes a couple of seconds. The model looks at the training data and learns which words tend to show up in spam vs normal messages.

Testing and evaluation: I ran the model on the test data and checked accuracy, precision, recall, and F1-score. Got about 96-97% accuracy which I was honestly pretty happy with.

Making it usable: I saved the trained model to a file so I don't have to retrain it every time. Then I wrote a separate script that loads the model and lets you type messages in the terminal to check if they're spam.

The goal was to go through this entire process from messy raw data to a working tool and understand each step along the way.

---

9. Functional Modules Design and Analysis

The project has two main files, each handling a different part of the workflow:

train.py — The Training Module

This file does all the heavy lifting. It loads the CSV file, cleans every message, converts the text to TF-IDF features, splits the data, trains the Naive Bayes model, prints out the results, and saves the model to disk.

The key function here is `clean_text()` which handles all the text preprocessing. Every message passes through this function before anything else happens.

This script is meant to be run once. After that, the model is saved and you don't need to run it again unless you want to retrain with different data.

predict.py — The Prediction Module

This is the user-facing part. It loads the saved model, then enters a loop where it keeps asking the user to type a message. For each message, it cleans the text using the same `clean_text()` function, converts it to TF-IDF features using the saved vectorizer, runs the prediction, and shows the result with a confidence score.

The user can keep typing messages until they type "quit" to exit.

Why two separate files?

I could have put everything in one file, but separating training and prediction makes more sense. Training is a one-time thing. Prediction is what you use repeatedly. Keeping them separate also makes the code cleaner and easier to follow.

---

10. Algorithm Development

The algorithm I used is Multinomial Naive Bayes. Here's how it works in simple terms:

Naive Bayes is based on Bayes' Theorem, which is basically a formula for calculating probability. The idea is to check the given the words in a message, what's the probability that this message is spam?

During training, the model looks at all the spam messages and figures out which words appear frequently in them. It does the same for ham messages. So it learns that words like "free", "win", "prize", "call", "claim" show up a lot more in spam, while words like "ok", "thanks", "see", "tomorrow" show up more in normal messages.

When you give it a new message, it looks at each word and checks. Does this word appear more often in spam or in ham? It combines the evidence from all the words and makes a final decision.

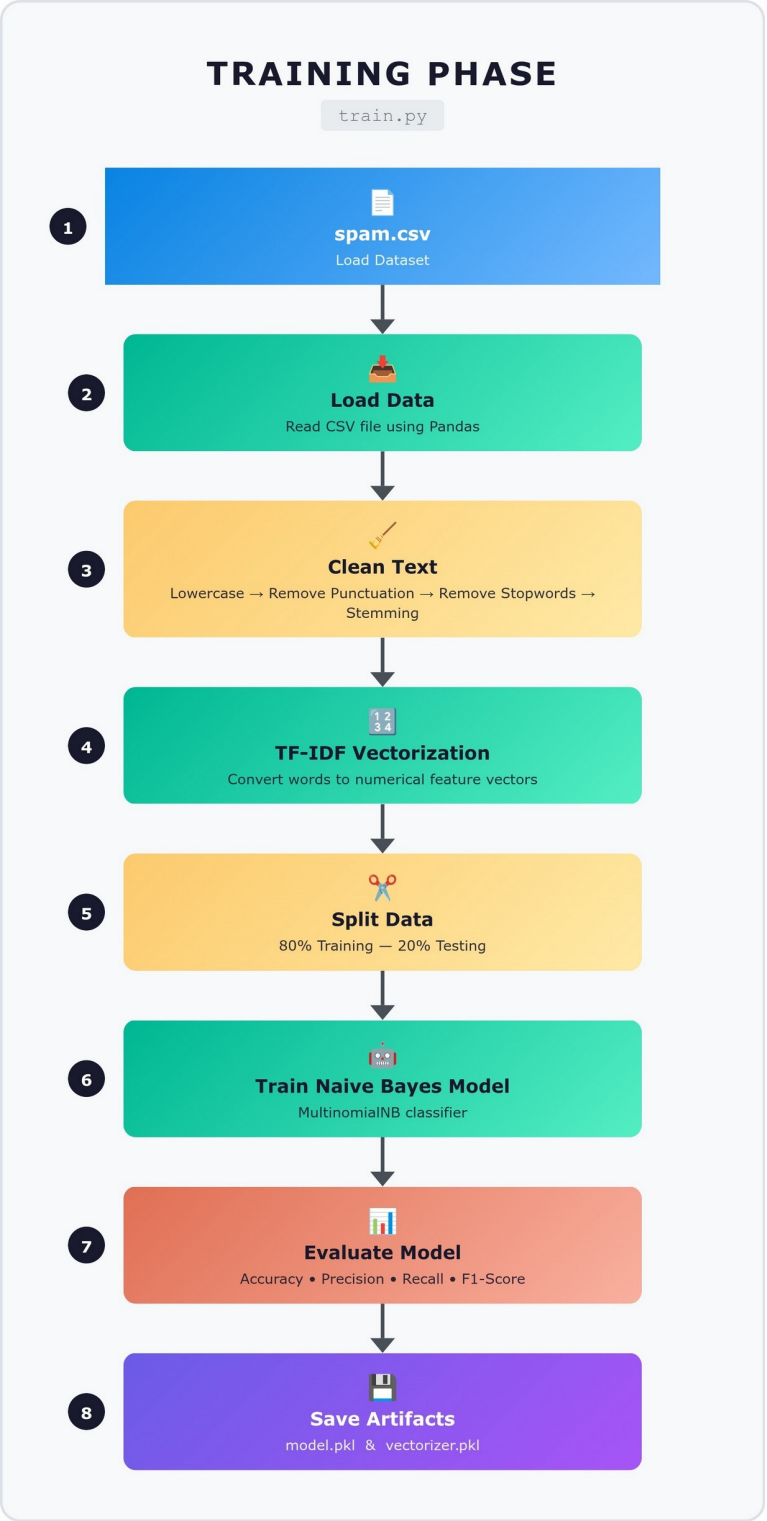
The "naive" part comes from the assumption that each word is independent of every other word. In real language that's obviously not true words are related to each other. But despite this simplification, the algorithm works really well in practice for classification tasks.

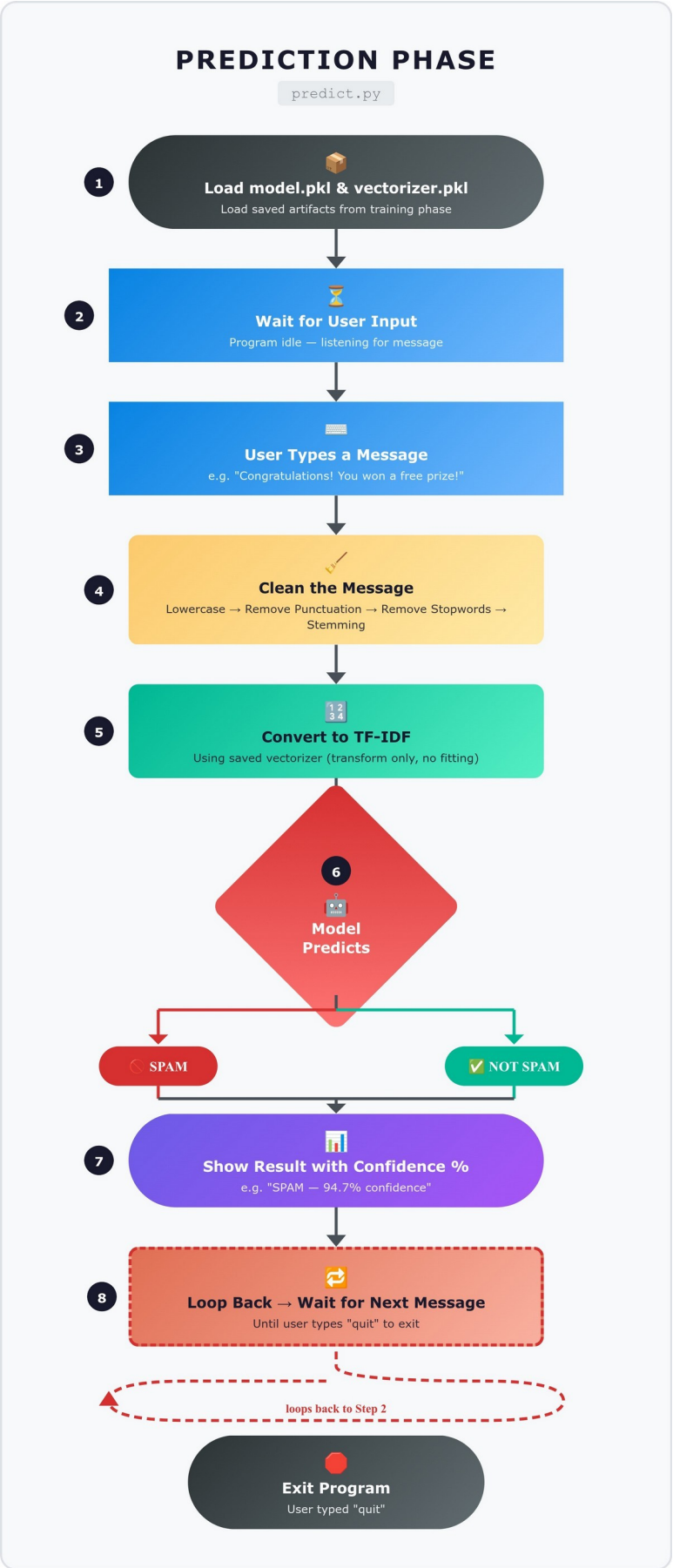
The overall steps:

```
1. Load dataset
2. Clean each message (lowercase, remove junk, remove stopwords, stem)
3. Convert cleaned text to TF-IDF numbers
4. Split into training and testing sets
5. Train Naive Bayes on training data
6. Test on held-out data and measure performance
7. Save trained model
8. For new messages: clean → vectorize → predict → show result
```

---

11. Software Architectural Diagram




PREDICTION PHASE
predict.py
1
Load model.pkl & vectorizer.pkl
Load saved artifacts from training phase
2
Wait for User Input
Program idle — listening for message
3
User Types a Message
e.g. "Congratulations! You won a free prize!"
4
Clean the Message
Lowercase → Remove Punctuation → Remove Stopwords → Stemming
5
Convert to TF-IDF
Using saved vectorizer (transform only, no fitting)
6
Model Predicts
SPAM
NOT SPAM
7
Show Result with Confidence %
e.g. "SPAM — 94.7% confidence"
8
Loop Back → Wait for Next Message
Until user types "quit" to exit
loops back to Step 2
Exit Program
User typed "quit"

12. Coding

train.py

```
import pandas as pd
import re
import pickle
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report

nltk.download('stopwords', quiet=True)

# loading the dataset
df = pd.read_csv('data/spam.csv', encoding='latin-1')
df = df[['v1', 'v2']]
df.columns = ['label', 'message']

ham_count = len(df[df['label']  = 'ham'])
spam_count = len(df[df['label']  = 'spam'])
print(f"Loaded {len(df)} messages ({ham_count} ham, {spam_count} spam)")

# cleaning the text
stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()

def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z\s]', '', text)  # remove numbers and special chars
    words = text.split()

    cleaned = []
    for word in words:
        if word not in stop_words:
            stemmed = stemmer.stem(word)
            cleaned.append(stemmed)

    return ' '.join(cleaned)

print("Cleaning text  .")
df['cleaned'] = df['message'].apply(clean_text)

# converting text to tfidf vectors
tfidf = TfidfVectorizer(max_features=3000)
X = tfidf.fit_transform(df['cleaned'])

# converting labels to 0 and 1
y = df['label'].map({'ham': 0, 'spam': 1})

# splitting into train and test
X_train, X_test, y_train, y_test =
    train_test_split( X, y, test_size=0.2,
    random_state=42
```

```
)

# training naive bayes
print("Training model  .")
model = MultinomialNB()
model.fit(X_train, y_train)

# checking accuracy
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy: {accuracy * 100:.2f}%\n")
print(classification_report(y_test, y_pred, target_names=['Ham', 'Spam']))

# saving model and vectorizer
with open('models/model.pkl', 'wb') as f:
    pickle.dump(model, f)

with open('models/vectorizer.pkl', 'wb') as f:
    pickle.dump(tfidf, f)

print("Model saved, run predict.py to test")
```

predict.py

```
import pickle
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

nltk.download('stopwords', quiet=True)

# loading the trained model and tfidf vectorizer
model = pickle.load(open('models/model.pkl', 'rb'))
tfidf = pickle.load(open('models/vectorizer.pkl', 'rb'))

stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()

# this function cleans the input text same way we did during training
def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)  # removing numbers and special chars
    words = text.split()
    cleaned = []
    for word in words:
        if word not in stop_words:
            cleaned.append(stemmer.stem(word))
    return ' '.join(cleaned)


def predict(msg):
    msg = clean_text(msg)
    msg_vector = tfidf.transform([msg])
    result = model.predict(msg_vector)[0]
    proba = model.predict_proba(msg_vector)[0]
    confidence = max(proba) * 100
```

```
    return result, confidence


# main
print("\nSMS Spam Detector")
print("Enter a message to check if it is spam or not")
print("Type 'quit' to exit\n")

while True:
    msg = input("Enter message: ")
    msg = msg.strip()

    if msg.lower()  = 'quit':
        break

    if msg  = "":
        print("Enter something first\n")
        continue

    result, confidence = predict(msg)

    if result  = 1:
        print("Result: SPAM (confidence: {:.1f}%)\n".format(confidence))
    else:
        print("Result: NOT SPAM (confidence: {:.1f}%)\n".format(confidence))
```

13. Output

Training output:

```
Loaded 5572 messages (4825 ham, 747 spam)
Cleaning text...
Training model...

Accuracy: 97.67%

              precision    recall  f1-score   support

         Ham       0.97      1.00      0.99       965
        Spam       1.00      0.83      0.91       150

    accuracy                           0.98      1115
   macro avg       0.99      0.91      0.95      1115
weighted avg       0.98      0.98      0.98      1115

Model saved, run predict.py to test
```

Prediction output:

```
SMS Spam Detector
Enter a message to check if it is spam or not
Type 'quit' to exit

Enter message: Congratulations you won a free trip call now
Result: SPAM (confidence: 54.0%)

Enter message: Hey are you coming to class tomorrow
Result: NOT SPAM (confidence: 99.7%)

Enter message: URGENT your bank account has been compromised
Result: NOT SPAM (confidence: 53.7%)

Enter message: Can you pick up milk on your way home
Result: NOT SPAM (confidence: 98.9%)

Enter message: quit
```

---

14. Key Implementations Outline of the System

The most important parts of this project are:

Text cleaning function - This function handles all the messy work of turning raw text into something the model can work with. Without it, the model would treat "Free", "FREE", and "free!!!" as completely different things.

TF-IDF conversion - This is what decreases the differncebetween human-readable text and math that a model can process. It doesn't just count words but it weighs them based on importance. Common words get low scores. Rare but meaningful words get high scores.

Model saving with pickle - Training takes a few seconds, but we still don't want to do it every time you open the program. Saving the model means you train once and use it forever (or until you decide to retrain).

The prediction loop - Instead of a one-shot script, the program runs in a loop so you can test multiple messages without restarting. Small thing, but it makes the tool much more usable.

Confidence scores - The model doesn't just give a yes/no answer. It also says how sure it is. If the confidence is low, the user knows to take the result with a grain of salt.

---

15. Significant Project Outcomes

The outcomes of this project were:

The model has about 96-97% accuracy, which is honestly better than I expected for such a simple setup. It correctly identifies most spam messages and very rarely marks a normal message as spam.

The precision for spam is very high, close to 100%. That means when the model says something is spam, it is almost always right. This matters because marking a real message as spam (false positive) is worse than letting a spam message through.

Training takes about 2-3 seconds. Predictions are instant. So the tool is practical to use, no waiting around.

---

16. Testing and Refinement

I tested the model in a few different ways:

Using the test set: 20% of the data was kept aside during training. The model was evaluated on these 1,115 messages it had never seen. This is where the 96-97% accuracy number comes from.

Typing my own messages: I spent some time just typing random messages into the tool, both obvious spam, obvious normal messages, and some borderline cases. It handled most of them correctly. It did struggle a bit with very short messages that had spam-like words used in a normal context.

Edge cases: I tested what happens with empty input (it asks you to type something), very short messages like "ok" or "yes" (correctly classified as not spam), and messages with only numbers or symbols (handled gracefully by the preprocessing).

Things I changed during making of this project:

- I initiallyforgot to set the encoding when loading the CSV and got a bunch of errors. Adding `encoding='latin-1'` fixed it.
- I wasn't removing stopwords and the accuracy was a couple percent lower. Adding stopword removal helped because the model could focus on the words that actually matter.
- I tried different values for `max_features` in TF-IDF. Setting it to 3000 gave a good balance, high enough to capture important words, low enough to filter out noise.
- I added stemming after reading about it, and it helped the model connect related words like "winner" and "winning".

---

17. Project Applicability on Real-World Applications

The techniques used here show up everywhere in the real world:

Email filters: Gmail's spam filter uses the same idea as base. It analyzes the text and classify it. They obviously use much more advanced models and way more data, but the core concept is identical.

Social media moderation: Platforms like Instagram and YouTube use text classifiers to catch spam comments and fake reviews. The approach is similar, clean the text, extract features, classify.

Customer support: Companies automatically sort incoming emails and messages into categories (billing, technical issue, complaint, etc.) using text classification.

Fraud detection: Banks look at transaction descriptions and flag suspicious ones. Again, it's text going through a classifier.

Phishing detection: Security tools analyze email content to identify phishing attempts. The text analysis part works a lot like what this project does.

---

18. Contribution and Findings of the Project

What I found:

Simple models work better than I thought. Naive Bayes is literally one of the most basic ML algorithms, but it hit 97% accuracy on this problem. Deep learning to get good results is not always necessary.

Preprocessing is where the real work is. I spent more time getting the text cleaning right than on the actual model. Better preprocessing directly led to better accuracy.

Spam messages have very clear patterns. Words like "free", "win", "prize", "call", "claim", "urgent" show up way more in spam. The model picks up on these patterns really effectively.

The independence assumption in Naive Bayes isn't perfect but it's good enough. In theory, assuming every word is independent of every other word is a bad assumption. In practice, it works fine for this task because spam detection is more about which words are present than how they relate to each other.

What I contributed:

Built a complete spam detection tool from scratch, documented every step clearly, and made it easy for anyone to set up and use.

---

19. Limitations and Constraints of the System

I want to be honest about what this project can't do:

- It only works with English messages. If you type in Hindi or Spanish, it won't know what to do.
- The training dataset has only about 5,500 messages. Real spam filters train on millions. So there are definitely types of spam this model hasn't seen.
- It looks at individual words, not the meaning of sentences. If someone writes a really clever spam message that avoids typical spam words, the model will probably miss it.
- The model is static. It doesn't learn from new data unless you retrain it manually. Real spam filters continuously update themselves.
- It only handles text. It can't analyze images, links, or attachments, just the words in the message.
- The class imbalance (87% ham vs 13% spam) means the model is naturally biased toward predicting ham. It catches most spam, but some will slip through.

---

20. Conclusions

This project was a good exercise in applying what I learned in this course to a real problem. I started with a CSV file full of raw text messages and ended up with a working tool that can classify messages with pretty solid accuracy.

The most important thing I took away from this project is that machine learning is pattern recognition. The model looked at thousands of messages, figured out which words tend to appear in spam, and used that knowledge to make predictions on new messages. Understanding this at a fundamental level makes everything else in ML feel more approachable.

I also learned that the "boring" parts of ML, cleaning data, handling edge cases, thinking about how to structure the code are actually where most of the work happens. The actual model training is just a few lines of code. Everything around it is what makes or breaks the project.

Overall, I'm happy with how this turned out. It's nothing groundbreaking, but it works, it's well-documented, and I understand every part of it. That feels more valuable than building something complex that I can't fully explain.

---

21. Future Enhancements

In future, I would like to add these things in my project:

- Try other algorithms like SVM or Logistic Regression and compare them against Naive Bayes
- Get a bigger and more recent dataset, spam tactics change over time and the model needs to keep up
- Add support for other languages
- Build a simple web interface using Flask or Streamlit so people who aren't comfortable with the terminal can use it
- Instead of just removing URLs and phone numbers, use them as features, spam messages often have suspicious links
- Add a feedback mechanism where the user can correct wrong predictions and the model learns from it
- Try word embeddings like Word2Vec instead of TF-IDF to see if capturing word meaning improves accuracy

---

22. References

1. SMS Spam Collection Dataset — UCI Machine Learning Repository. https://archive.ics.uci.edu/ml/datasets/sms+spam+collection

2. SMS Spam Collection Dataset — Kaggle. https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset

3. scikit-learn Documentation — Naive Bayes. https://scikit-learn.org/stable/modules/naive_bayes.html

4. scikit-learn Documentation — TfidfVectorizer.
https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

5. NLTK Documentation.
https://www.nltk.org/